

LAPORAN WORKSHOP KECERDASAN BUATAN

PERTEMUAN 3



Dicho Dwi Putra Setyawan

5324600013

PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI REKAYASA MULTIMEDIA
JURUSAN TEKNOLOGI MULTIMEDIA KREATIF
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

I. Materi

-

II. Alat & Bahan

- A. Laptop
- B. Microsoft Word
- C. Visual Studio Code

III. Tugas

A. TUGAS PENDAHULUAN

1. EXAMPLE 3.1 THE SVM.PY PROGRAM

```
##Example 3.1 Python SVM Classifications
from sklearn import svm
X = [[170, 70, 10], [180, 80,12], [170, 65,
8],[160, 55, 7]]
#Height[cm], Weight[kg], Shoesize[UK]
y = [0, 0, 1, 1] #Gender, 0: Male, 1: Female
clf = svm.SVC()
clf.fit(X, y)
#Predict
p = clf.predict([[160, 60, 7]])
print(p)
```

[1]

PS D:\Tugas\Semester 5\AI\MasterHub> █

a. EXERCISE 3.1

Modify the Python program from Example 3.1 to use six samples of X and y.

```
# Penyelesaian Exercise 3.1
from sklearn import svm

# Menambahkan 2 sampel baru sehingga total
menjadi 6 sampel
# Format: Height [cm], Weight [kg], Shoesize
[UK]
X = [
    [170, 70, 10],
    [180, 80, 12],
    [170, 65, 8],
    [160, 55, 7],
```

```

        [175, 73, 10], # Tambahan 1 (Misal:
Laki-laki)
        [155, 48, 5]    # Tambahan 2 (Misal:
Perempuan)
    ]

    # Gender, 0: Male, 1: Female
    y = [0, 0, 1, 1, 0, 1] # Menambahkan label
    0 dan 1 untuk sampel baru

    clf = svm.SVC()
    clf.fit(X, y)

    #Predict
    p = clf.predict([[160, 60, 7]])
    print ("Hasil prediksi:", p)

```

```

Hasil prediksi: [1]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

2. EXAMPLE 3.2 THE SVM2.PY PROGRAM

```

#Example 3.2 Python SVM Iris Classifications
from sklearn import svm, datasets

iris = datasets.load_iris()
# Take the first two features: Sepal length and
Sepal width
X = iris.data[:, :2]
y = iris.target
#0: Setosa, 1: Versicolour, 2: Virginica
print (y)

clf = svm.SVC()
clf.fit(X, y)

#Predict the flower for a given Sepal length and
width
p = clf.predict([[5.4, 3.2]])
print (p)

```



```

clf.fit(X, y)
# Predict the flower for a given Sepal length and
width
p = clf.predict([[5.4, 3.2]])
print(p)

```

```

[0]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

4. EXAMPLE 3.4 THE SVM4.PY PROGRAM

```

#Example 3.4 Python SVM Iris URL Classifications
from sklearn import svm, datasets
import pandas as pd
from matplotlib import pyplot
from pandas.plotting import scatter_matrix

df = pd.read_csv('https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv')

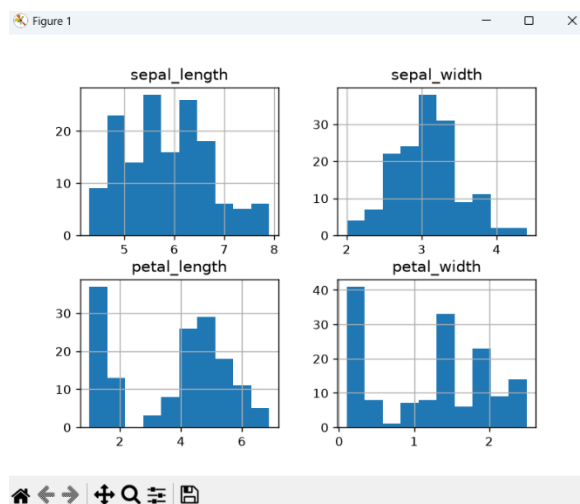
print(df.shape)
print(df.head(10))
print(df.tail(10))
print(df.describe())
#count NAN
print(df.isna().sum().sum())
#drop NAN values
df = df.dropna()
print(df.groupby('species').size())
# histograms
df.hist()
pyplot.show()
# scatter plot matrix
scatter_matrix(df)
pyplot.show()
X = df.values[:, :2]
s = df['species']
d = dict([(y, x) for x, y in enumerate(sorted(set(s)))])
y = [d[x] for x in s]
clf = svm.SVC()
clf.fit(X, y)

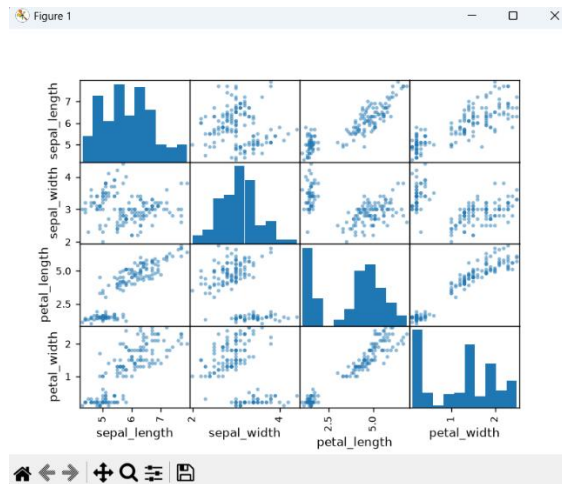
```

```
#Predict the flower for a given Sepal length and
width
p = clf.predict([[5.4, 3.2]])
pprint (p)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
7	5.0	3.4	1.5	0.2	setosa
8	4.4	2.9	1.4	0.2	setosa
9	4.9	3.1	1.5	0.1	setosa
140	6.7	3.1	5.6	2.4	virginica
141	6.9	3.1	5.1	2.3	virginica
142	5.8	2.7	5.1	1.9	virginica
143	6.8	3.2	5.9	2.3	virginica
144	6.7	3.3	5.7	2.5	virginica
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica

143	6.8	3.2	5.9	2.3	virginica
144	6.7	3.3	5.7	2.5	virginica
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica
	sepal_length	sepal_width	petal_length	petal_width	
count	150.000000	150.000000	150.000000	150.000000	
mean	5.843333	3.054000	3.758667	1.198667	
std	0.828066	0.433594	1.764420	0.763161	
min	4.300000	2.000000	1.000000	0.100000	
25%	5.100000	2.800000	1.600000	0.300000	
50%	5.800000	3.000000	4.350000	1.300000	
75%	6.400000	3.300000	5.100000	1.800000	
max	7.900000	4.400000	6.900000	2.500000	
0					
species					
setosa	50				
versicolor	50				
virginica	50				
dtype:	int64				





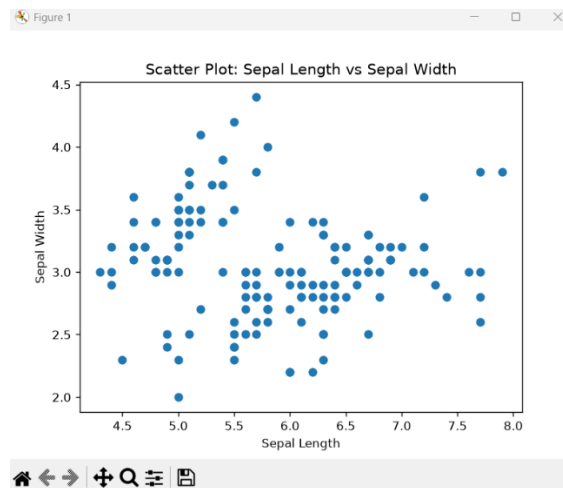
a. EXERCISE 3.3

Modify the Python program from Example 3.4 so that it plots the first two features (sepal length and sepal width) of all the data points as a scatter plot.

```
# Penyelesaian Exercise 3.3
import pandas as pd
from matplotlib import pyplot as plt

df = pd.read_csv('https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv')

# Memplot sepal length vs sepal width sebagai scatter plot
plt.scatter(df['sepal_length'], df['sepal_width'])
plt.title('Scatter Plot: Sepal Length vs Sepal Width')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.show()
```



5. EXAMPLE 3.5 Breast Cancer Data

```
#Example 3.5 Breast Cancer Data
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
print(cancer['feature_names'])
print(cancer['data'])
print(cancer.target_names)
```

```
['mean radius' 'mean texture' 'mean perimeter' 'mean area'
 'mean smoothness' 'mean compactness' 'mean concavity'
 'mean concave points' 'mean symmetry' 'mean fractal dimension'
 'radius error' 'texture error' 'perimeter error' 'area error'
 'smoothness error' 'compactness error' 'concavity error'
 'concave points error' 'symmetry error' 'fractal dimension error'
 'worst radius' 'worst texture' 'worst perimeter' 'worst area'
 'worst smoothness' 'worst compactness' 'worst concavity'
 'worst concave points' 'worst symmetry' 'worst fractal dimension']
[[1.799e+01 1.038e+01 1.228e+02 ... 2.654e-01 4.601e-01 1.189e-01]
 [2.057e+01 1.777e+01 1.329e+02 ... 1.860e-01 2.750e-01 8.902e-02]
 [1.969e+01 2.125e+01 1.300e+02 ... 2.430e-01 3.613e-01 8.758e-02]
 ...
 [1.660e+01 2.808e+01 1.083e+02 ... 1.418e-01 2.218e-01 7.820e-02]
 [2.060e+01 2.933e+01 1.401e+02 ... 2.650e-01 4.087e-01 1.240e-01]
 [7.760e+00 2.454e+01 4.792e+01 ... 0.000e+00 2.871e-01 7.039e-02]]
['malignant' 'benign']
PS D:\Tugas\Semester 5\AI\MasterHub> []
```

6. EXAMPLE 3.6 Python SVM Breast Cancer Classifications

```
#Example 3.6 Python SVM Breast Cancer
Classifications
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
cancer = load_breast_cancer()
X = cancer.data # All of the features
y = cancer.target # All of the labels
```

```

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size =
0.2, random_state = 20)
clf = SVC()
clf.fit(X_train, y_train)
#Prediction
y_predict = clf.predict(X_test)
#Print Confusion Matrix and Classification
Report
from sklearn.metrics import
classification_report, confusion_matrix
cm = np.array(confusion_matrix(y_test,
y_predict, labels=[1,0]))
confusion = pd.DataFrame(cm, index=['is_cancer',
'is_healthy'],
columns=['predicted_cancer','predicted_
healthy'])
print(confusion)
print(classification_report(y_test, y_predict))

```

	predicted_cancer	predicted_healthy		
is_cancer	66	0		
is_healthy	8	40		
	precision	recall	f1-score	support
0	1.00	0.83	0.91	48
1	0.89	1.00	0.94	66
accuracy			0.93	114
macro avg	0.95	0.92	0.93	114
weighted avg	0.94	0.93	0.93	114

PS D:\Tugas\Semester 5\AI\MasterHub> █

a. EXERCISE 3.4

Modify the Python program from Example 3.6 so that it plots the histogram of radius, size, texture, and smoothness of all the data points.

```

#Example 3.6 Python SVM Breast Cancer
Classifications
import pandas as pd
import numpy as np
from sklearn.datasets import
load_breast_cancer
from sklearn.model_selection import
train_test_split
from sklearn.svm import SVC

```

```

from sklearn.metrics import
classification_report, confusion_matrix

cancer = load_breast_cancer()
X = cancer.data # All of the features
y = cancer.target # All of the labels

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size = 0.2,
random_state = 20)

clf = SVC()
clf.fit(X_train, y_train)

#Prediction
y_predict = clf.predict(X_test)

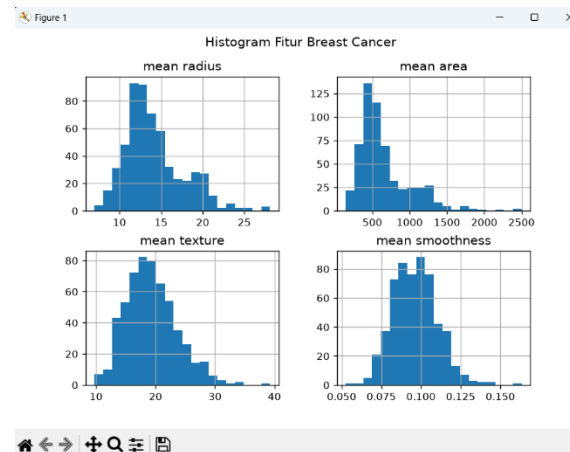
#Print Confusion Matrix and Classification
Report
cm = np.array(confusion_matrix(y_test,
y_predict, labels=[1,0]))
confusion = pd.DataFrame(cm,
index=['is_cancer', 'is_healthy'],
columns=['predicted_cancer',
'predicted_healthy'])
print (confusion)
print (classification_report(y_test,
y_predict # Penyelesaian Exercise 3.4
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import
load_breast_cancer

cancer = load_breast_cancer()
# Mengonversi data ke DataFrame agar lebih
mudah diplot
df_cancer = pd.DataFrame(cancer.data,
columns=cancer.feature_names)

# Memilih fitur spesifik sesuai instruksi:
radius, area (size), texture, smoothness
features_to_plot = ['mean radius', 'mean
area', 'mean texture', 'mean smoothness']

```

```
# Membuat histogram
df_cancer[features_to_plot].hist(figsize=(
10, 8), bins=20)
plt.suptitle('Histogram      Fitur      Breast
Cancer')
plt.show()
```



7. EXAMPLE 3.7 Naive Bayes Iris

```
#Example 3.7 Naive Bayes Iris
from sklearn.datasets import load_iris
from      sklearn.model_selection      import
train_test_split
from sklearn.naive_bayes import GaussianNB
X, y = load_iris(return_X_y=True)
print(X)
clf = GaussianNB()
clf.fit(X, y)
p = clf.predict([[5.0, 3.4, 1.5, 0.4]])
print(p)
```

```
[6.9 3.1 5.1 2.3]
[5.8 2.7 5.1 1.9]
[6.8 3.2 5.9 2.3]
[6.7 3.3 5.7 2.5]
[6.7 3.  5.2 2.3]
[6.3 2.5 5.  1.9]
[6.5 3.  5.2 2. ]
[6.2 3.4 5.4 2.3]
[5.9 3.  5.1 1.8]]
[0]
PS D:\Tugas\Semester 5\AI\MasterHub> █
```

a. EXERCISE 3.5

Modify the Python program from Example 3.7 so that it first trains the naive Bayes model, saves the model to a file, loads the model from the file, and makes a prediction with the model.

```
# Penyelesaian Exercise 3.5
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
import joblib # Menggunakan joblib untuk
menyimpan dan memuat model

X, y = load_iris(return_X_y=True)

# 1. Melatih Model
clf = GaussianNB()
clf.fit(X, y)

# 2. Menyimpan model ke sebuah file
(Serialization)
joblib.dump(clf, 'naive_bayes_model.pkl')
print("Model berhasil disimpan ke
'naive_bayes_model.pkl'")

# 3. Memuat model dari file
(Deserialization)
clf_loaded =
joblib.load('naive_bayes_model.pkl')

# 4. Membuat prediksi dengan model yang
telah dimuat
p = clf_loaded.predict([[5.0, 3.4, 1.5,
0.4]])
print("Hasil prediksi dari model yang
dimuat:", p)
```

```
Model berhasil disimpan ke 'naive_bayes_model.pkl'
Hasil prediksi dari model yang dimuat: [0]
PS D:\Tugas\Semester 5\AI\MasterHub> █
```

8. EXAMPLE 3.8 LinearDiscriminantAnalysis Classification

```
#Example 3.8 LinearDiscriminantAnalysis
Classification
from sklearn.datasets import load_iris
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
```

```
X, y = make_classification(n_samples=1000,
n_features=4,
n_informative=2, n_redundant=0,
random_state=0, shuffle=False)
print(X)
clf = LinearDiscriminantAnalysis()
clf.fit(X, y)
print(clf.predict([[0, 0, 0, 0]]))
```

```
[[ -1.66853167 -1.29901346  0.2746472  -0.60362044]
 [ -2.9728827  -1.08878294  0.70885958  0.42281857]
 [ -0.59614125 -1.37007001 -3.11685659  0.64445203]
 ...
 [  0.91711204  1.10596645  0.86766522 -2.25625012]
 [  0.10027664  1.45875846 -0.44360274 -0.67002328]
 [  1.0415229  -0.01987143  0.15216419 -1.9405334  ]]
[1]
PS D:\Tugas\Semester 5\AI\MasterHub> █
```

a. EXERCISE 3.6

Modify the Python program from Example 3.8 so that it uses 6 features and 2,000 samples. Compare the results with Example 3.8.

```
# Penyelesaian Exercise 3.5
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
import joblib # Menggunakan joblib untuk
menyimpan dan memuat model

X, y = load_iris(return_X_y=True)

# 1. Melatih Model
clf = GaussianNB()
clf.fit(X, y)

# 2. Menyimpan model ke sebuah file
(Serialization)
joblib.dump(clf, 'naive_bayes_model.pkl')
print("Model berhasil disimpan ke
'naive_bayes_model.pkl'")

# 3. Memuat model dari file
(Deserialization)
clf_loaded =
joblib.load('naive_bayes_model.pkl')
```

```
# 4. Membuat prediksi dengan model yang
telah dimuat
p = clf_loaded.predict([[5.0, 3.4, 1.5,
0.4]])
print("Hasil prediksi dari model yang
dimuat:", p) # Penyelesaian Exercise 3.6
from sklearn.datasets import
make_classification
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis

# Mengubah n_samples menjadi 2000 dan
n_features menjadi 6
X, y = make_classification(n_samples=2000,
n_features=6, n_informative=2,
n_redundant=0, random_state=0,
shuffle=False)

clf = LinearDiscriminantAnalysis()
clf.fit(X, y)

# Karena ada 6 fitur, input prediksi juga
harus memiliki 6 nilai
print("Hasil prediksi LDA (2000 sampel, 6
fitur):", clf.predict([[0, 0, 0, 0, 0, 0]]))
```

```
Hasil prediksi LDA (2000 sampel, 6 fitur): [0]
PS D:\Tugas\Semester 5\AI\MasterHub>
```

9. EXAMPLE 3.9 Principal Component Analysis

```
#Example 3.9 Principal Component Analysis
from sklearn.datasets import load_iris
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
X, y = make_classification(n_samples=1000,
n_features=4,
n_informative=2, n_redundant=0,
random_state=0, shuffle=False)
print(X)
clf = PCA()
clf.fit(X, y)
print(clf.explained_variance_ratio_)
print(clf.singular_values_)
```

```

[[-1.66853167 -1.29901346  0.2746472 -0.60362044]
 [-2.9728827  -1.08878294  0.70885958  0.42281857]
 [-0.59614125 -1.37007001 -3.11685659  0.64445203]
 ...
 [ 0.91711204  1.10596645  0.86766522 -2.25625012]
 [ 0.10027664  1.45875846 -0.44360274 -0.67002328]
 [ 1.0415229  -0.01987143  0.15216419 -1.9405334  ]]
[0.35054565 0.26955436 0.1920725  0.1878275  ]
[42.31488145 37.10598956 31.32228453 30.97422379]
PS D:\Tugas\Semester 5\AI\MasterHub>

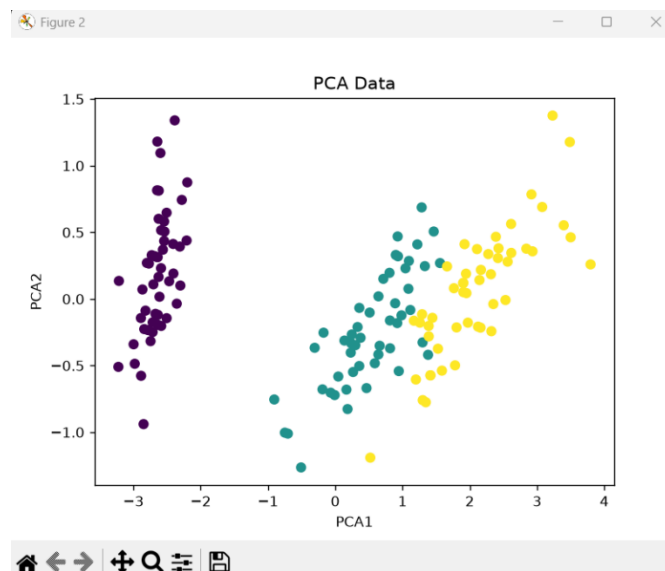
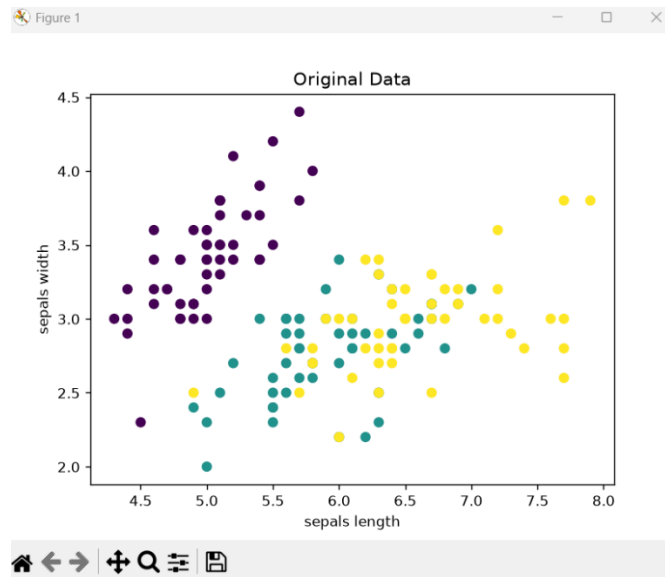
```

10. EXAMPLE 3.10 Principal Component Analysis Iris

```

#Example 3.10 Principal Component Analysis Iris
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn import decomposition
from sklearn import datasets
#Load Iris data
iris = datasets.load_iris()
X = iris.data
y = iris.target
#Plot Iris data
f = plt.figure(1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.xlabel('sepals length')
plt.ylabel('sepals width')
plt.title('Original Data')
f.show()
#Perform PCA
pca = decomposition.PCA(n_components=3)
pca.fit(X)
X1 = pca.transform(X)
#Plot PCA data
g = plt.figure(2)
#y = np.choose(y, [1, 2, 0]).astype(float)
plt.scatter(X1[:, 0], X1[:, 1], c=y)
plt.xlabel('PCA1')
plt.ylabel('PCA2')
plt.title('PCA Data')
g.show()

```



a. EXERCISE 3.7

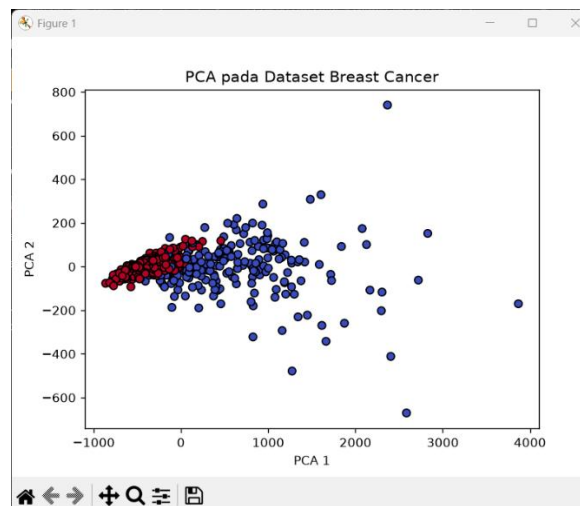
Modify the Python program from Example 3.10 so that it performs PCA on the breast cancer data.

```
# Penyelesaian Exercise 3.7
import matplotlib.pyplot as plt
from sklearn import decomposition
from sklearn.datasets import load_breast_cancer

# Muat data breast cancer
cancer = load_breast_cancer()
X = cancer.data
y = cancer.target
```

```
# Lakukan PCA untuk mereduksi menjadi 2
komponen utama (agar mudah diplot 2D)
pca = decomposition.PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Plot Data PCA
plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y,
            cmap='coolwarm', edgecolor='k')
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('PCA pada Dataset Breast Cancer')
plt.show()
```



11. EXAMPLE 2.11: THE INPUT2.PY PROGRAM

```
#Example 3.11 Decision Tree Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
N = y_test.shape[0]
C = (y_test == y_pred).sum()
print("Total points: %d Correctly labeled points : %d" % (N,C))
```

```
Total points: 75 Correctly labeled points : 72
PS D:\Tugas\Semester 5\AI\MasterHub>
```

a. EXERCISE 3.8

Modify the Python program from Example 3.11 so that it performs decision tree classification on Scikit-Learn's wine data (https://scikitlearn.org/stable/modules/generated/sklearn.dataset.s.load_wine.html).

```
# Penyelesaian Exercise 3.8
from sklearn.datasets import load_wine
from sklearn.model_selection import
train_test_split
from sklearn.tree import
DecisionTreeClassifier

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.5,
random_state=0)

clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

N = y_test.shape[0]
C = (y_test == y_pred).sum()
print("Total poin: %d | Poin berlabel benar:
%d" % (N, C))
```

```
Total poin: 89 | Poin berlabel benar: 80
PS D:\Tugas\Semester 5\AI\MasterHub>
```

12. EXAMPLE 3.12 Random Forest

```
#Example 3.12 Random Forest
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split
from sklearn.ensemble import
RandomForestClassifier
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_
size=0.5, random_state=0)
```

```

clf = RandomForestClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points
: %d" %(y_test.
shape[0], (y_test == y_pred).sum()))

```

```

Total points: 75 Correctly labeled points : 71
PS D:\Tugas\Semester 5\AI\MasterHub>

```

a. EXERCISE 3.9

Modify the Python program from Example 3.12 so that it performs random forest classification on Scikit-Learn's diabetes data(https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html#sklearn.datasets.load_diabetes).

```

# Penyelesaian Exercise 3.9
from sklearn.datasets import load_diabetes
from sklearn.model_selection import
train_test_split
from sklearn.ensemble import
RandomForestRegressor

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.5,
random_state=0)

reg = RandomForestRegressor(random_state=0)
reg.fit(X_train, y_train)
score = reg.score(X_test, y_test) # R-
squared untuk regresi
print("Skor R-squared (Random Forest
Regressor) pada data diabetes: %.2f" %
score)

```

```

Skor R-squared (Random Forest Regressor) pada data diabetes: 0.32
PS D:\Tugas\Semester 5\AI\MasterHub>

```

13. EXAMPLE 3.13 K-Nearest Neighbors Classification

```

#Example 3.13 K-Nearest Neighbors Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split

```

```

from sklearn.neighbors import
KNeighborsClassifier
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_
size=0.5, random_state=0)
clf = KNeighborsClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Total points: %d Correctly labeled points
: %d" %(y_test.
shape[0], (y_test == y_pred).sum()))

```

```

Total points: 75 Correctly labeled points : 72
PS D:\Tugas\Semester 5\AI\MasterHub>

```

14. EXAMPLE 3.14 Comparison of Different Classification

```

#Example 3.14 Comparison of Different
Classification
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import
RandomForestClassifier
from sklearn.neighbors import
KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.discriminant_analysis import
QuadraticDiscriminantAnalysis
names = [ "SVM", "Naive Bayes", "LDA",
"QDA", "Decision Tree", "Random Forest",
"Nearest Neighbors", "Neural Networks"]
classifiers = [
SVC(),
GaussianNB(),
LinearDiscriminantAnalysis(),
QuadraticDiscriminantAnalysis(),
DecisionTreeClassifier(),
RandomForestClassifier(),

```

```

KNeighborsClassifier(),
MLPClassifier(alpha=1, max_iter=1000)]
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.5,
random_state=0)
for name, clf in zip(names, classifiers):
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)
    print(name + ": " + str(score))

```

```

SVM: 0.9466666666666667
Naive Bayes: 0.9466666666666667
LDA: 0.96
QDA: 0.96
Decision Tree: 0.96
Random Forest: 0.96
Nearest Neighbors: 0.96
Neural Networks: 0.9866666666666667
PS D:\Tugas\Semester 5\AI\MasterHub> █

```

a. EXERCISE 3.10

Modify the Python program from Example 3.14 so that it performs the classification on Scikit-Learn's diabetes data (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html#sklearn.datasets.load_diabetes).

```

# Penyelesaian Exercise 3.10
from sklearn.datasets import load_diabetes
from sklearn.model_selection import
train_test_split
from sklearn.svm import SVR
from sklearn.tree import
DecisionTreeRegressor
from sklearn.ensemble import
RandomForestRegressor
from sklearn.neighbors import
KNeighborsRegressor

names = ["SVM Regressor", "Decision Tree",
"Random Forest", "Nearest Neighbors"]
regressors = [SVR(),
DecisionTreeRegressor(),

```

```

RandomForestRegressor(),
KNeighborsRegressor()]

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.5,
random_state=0)

for name, reg in zip(names, regressors):
    reg.fit(X_train, y_train)
    score = reg.score(X_test, y_test)
    print(name + ": " + str(score))

```

```

SVM Regressor: 0.0964032518489577
Decision Tree: -0.2606313271918992
Random Forest: 0.33452390081791383
Nearest Neighbors: 0.3199078093887773
PS D:\Tugas\Semester 5\AI\MasterHub>

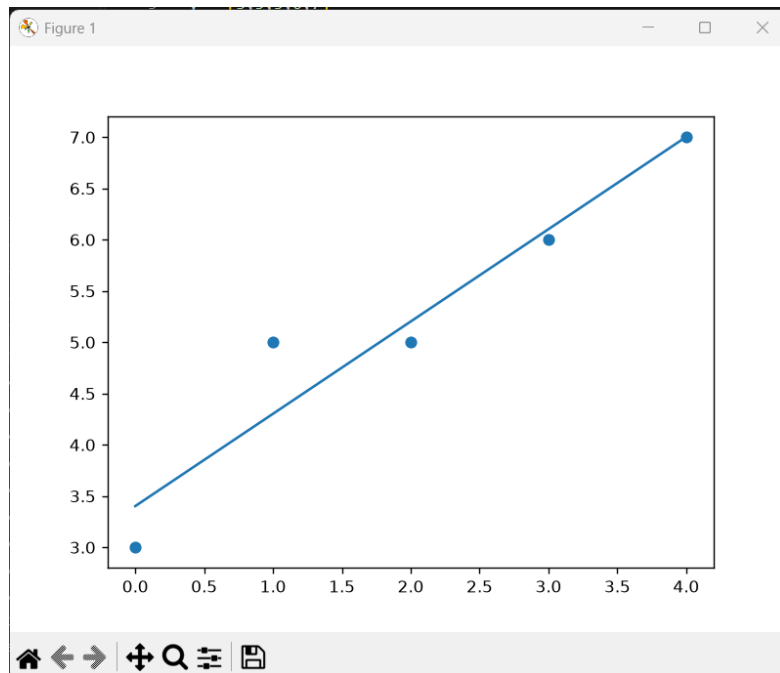
```

15. EXAMPLE 3.15 Linear Regression

```

#Example 3.15 Linear Regression
import matplotlib.pyplot as plt
from scipy import stats
x = [0,1,2,3,4]
y = [3,5,5,6,7]
slope, intercept, r, p, std_err =
stats.linregress(x, y)
print("slope: ", slope)
print("intercept: ", intercept)
def myfunc(x):
    return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()

```



a. EXERCISE 3.11

Modify the Python program from Example 3.15 so that it has more data points in x and y. Also, add an x label, y label, title, legend, and grids to the plot.

```
# Penyelesaian Exercise 3.11
import matplotlib.pyplot as plt
from scipy import stats

# Menambah titik data
x = [0, 1, 2, 3, 4, 5, 6, 7]
y = [3, 5, 5, 6, 7, 8, 9, 10]

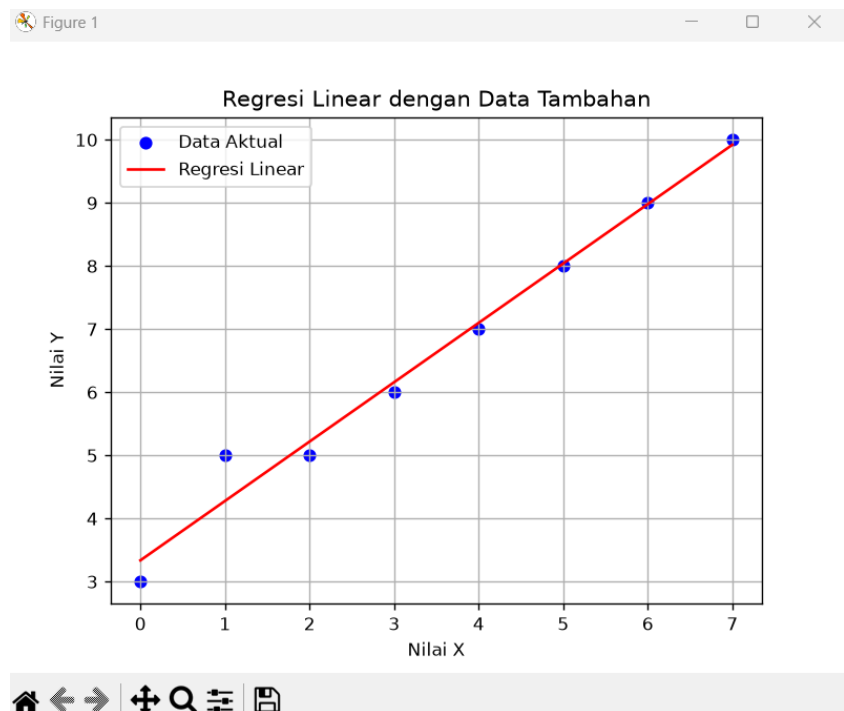
slope, intercept, r, p, std_err = stats.linregress(x, y)

def myfunc(x_val):
    return slope * x_val + intercept

mymodel = list(map(myfunc, x))

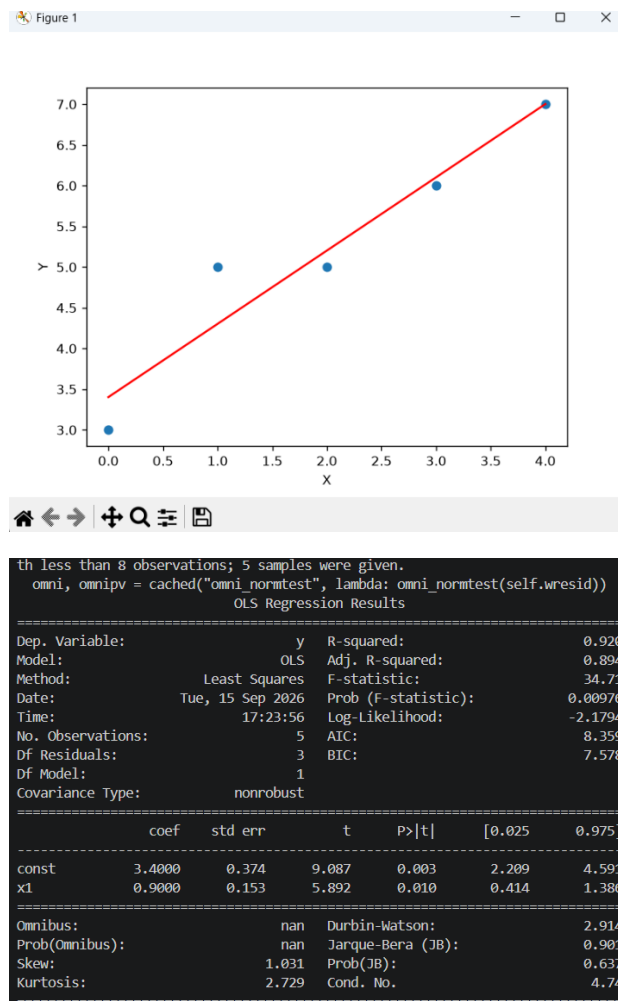
plt.scatter(x, y, label='Data Aktual', color='blue')
plt.plot(x, mymodel, label='Regresi Linear', color='red')
plt.xlabel('Nilai X')
plt.ylabel('Nilai Y')
```

```
plt.title('Regresi Linear dengan Data Tambahan')
plt.legend()
plt.grid(True)
plt.show()
```



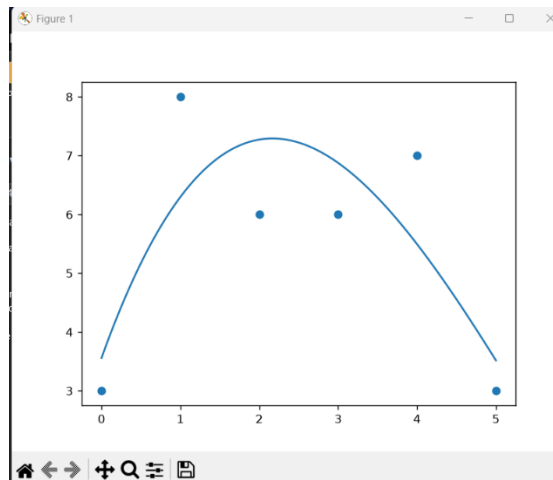
16. EXAMPLE 3.15A Linear Regression

```
#Example 3.15a Linear Regression
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
x = [0,1,2,3,4]
y = [3,5,5,6,7]
x1=sm.add_constant(x)
model = sm.OLS(y,x1)
results = model.fit()
print (results.params)
print (results.summary())
y_pred=results.predict(x1)
plt.scatter(x,y)
plt.xlabel("X")
plt.ylabel("Y")
plt.plot(x,y_pred, "r")
plt.show()
```



17. EXAMPLE 3.16 Polynomial Regression

```
#Example 3.16 Polynomial Regression
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np
x = [0,1,2,3,4,5]
y = [3,8,6,6,7,3]
mymodel = np.poly1d(np.polyfit(x, y, 3))
print(mymodel)
myline = np.linspace(0, 5, 100)
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```



18. EXAMPLE 3.17 Least Squares Fitting

```
#Example 3.17 Least Squares Fitting
import numpy as np
import scipy.optimize as optimization
x = np.array([0,1,2,3,4,5])
y = np.array([100,90,60,30,10,1])
def func(x, a, b, c):
    return a * np.exp(-b * x) + c
popt, pcov = optimization.curve_fit(func, x, y)
print ("Best fit a b c: ",popt)
print ("Best fit covariance: ",pcov)
```

```
return a * np.exp(-b * x) + c
Best fit a b c: [ 4.09564521e+02  6.21465806e-02 -3.04106842e+02]
Best fit covariance: [[ 4.92893733e+05 -8.73855445e+01 -4.95831373e+05]
 [-8.73855445e+01  1.55361514e-02  8.79419359e+01]
 [-4.95831373e+05  8.79419359e+01  4.98827106e+05]]
PS D:\Tugas\Semester 5\AI\MasterHub>
```

19. EXAMPLE 3.18 Multiple Linear Regression

```
#Example 3.18 Multiple Linear Regression
from sklearn import linear_model
import numpy as np
x =
np.array([[0,3,5],[1,4,6],[2,5,7],[3,6,8],[4,7,9]])
y = np.array([3,5,5,6,7])
reg = linear_model.LinearRegression()
reg.fit(x, y)
print('Coefficients: \n', reg.coef_)
print('Intercept: \n', reg.intercept_)
pred = reg.predict([[5,8,10]])
print('Predition: \n', pred)
```

```

Coefficients:
 [0.3 0.3 0.3]
Intercept:
 0.9999999999999991
Prediction:
 [7.9]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

a. EXERCISE 3.12

Modify the Python program from Example 3.18 so that it performs multiple linear regression on Scikit-Learn's Linnerrud dataset; see the following: https://scikit-learn.org/stable/datasets/toy_dataset.html#linnerrud-dataset https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_linnerud.html.

```

# Penyelesaian Exercise 3.12
from sklearn import linear_model
from sklearn.datasets import load_linnerud

# Muat data Linnerrud (fisiologis
memprediksi atribut olahraga)
linnerrud = load_linnerud()
X = linnerrud.data
y = linnerrud.target # Multiple output

reg = linear_model.LinearRegression()
reg.fit(X, y)

print('Coefficients:\n', reg.coef_)
print('Intercept:\n', reg.intercept_)

```

```

Coefficients:
[[-0.47502636 -0.21771647  0.09308837]
 [-0.13687023 -0.04033662  0.0279736 ]
 [ 0.00107079  0.04202941 -0.02946117]]
Intercept:
[208.23351881  40.59787542  52.04362105]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

20. EXAMPLE 3.19 Logistic Regression

```

#Example 3.19 Logistic Regression
import numpy as np
from sklearn.linear_model import
LogisticRegression
X = np.array([[0],[1],[2],[3],[4],[5]])
y = np.array([1,2,3,30,32,31])

```

```
clf = LogisticRegression(random_state=0).fit(X,
y)
print(clf.predict([[6]]))
print(clf.predict_proba([[6]]))
print(clf.score(X, y))
```

```
[31]
[[0.00111396 0.00810048 0.03285548 0.09767681 0.61033535 0.24991792]]
0.6666666666666666
PS D:\Tugas\Semester 5\AI\MasterHub>
```

21. EXAMPLE 3.20 K-means Clustering

```
Example 3.20 K-means Clustering
from sklearn.cluster import KMeans
import numpy as np
X = np.array([[1, 2, 3], [1, 4, 2], [1, 0, 3], \
[10, 2, 4], [9, 4, 3], [11, 0, 2]])
kmeans = KMeans(n_clusters=2,
random_state=0).fit(X)
print(kmeans.labels_)
print(kmeans.cluster_centers_)
print(kmeans.predict([[12, 3, 1]]))
```

```
[1 1 1 0 0 0]
[[10.      2.      3.      ]
 [ 1.      2.      2.6666667]]
[0]
PS D:\Tugas\Semester 5\AI\MasterHub>
```

a. EXERCISE 3.13

Modify the Python program from Example 3.20 so that it uses Scikit-Learn's function, `sklearn.datasets.make_blobs()`, to generate sample data points for K-means clustering. See the following links for the `sklearn.datasets.make_blobs()`: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html https://scikit-learn.org/stable/datasets/sample_generators.html#sample-generators.

```
# Penyelesaian Exercise 3.13
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

# Menghasilkan data dengan make_blobs
```

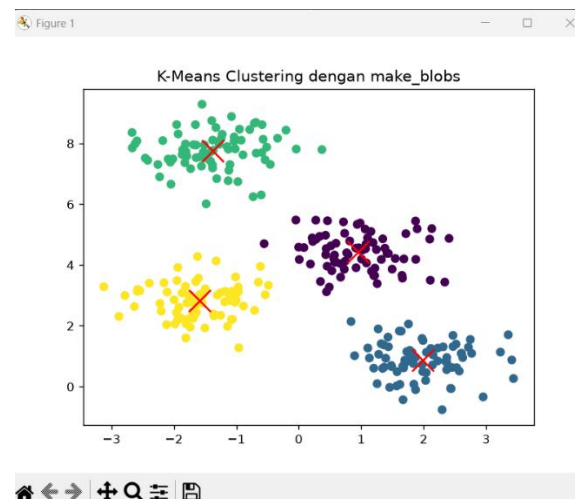
```

X, y_true = make_blobs(n_samples=300,
                        centers=4,
                        cluster_std=0.60,
                        random_state=0)

kmeans = KMeans(n_clusters=4,
                random_state=0).fit(X)

plt.scatter(X[:, 0], X[:, 1],
            c=kmeans.labels_, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1],
            s=300,
            c='red', marker='x')
plt.title('K-Means Clustering dengan
            make_blobs')
plt.show()

```



22. EXAMPLE 3.21 Semi-supervised Learning

```

#Example 3.21 Semi-supervised Learning
import numpy as np
import matplotlib.pyplot as plt
from sklearn.semi_supervised import
LabelSpreading
from sklearn.datasets import make_circles
X =
np.array([[0,1],[1,1],[2,0],[3,1],[10,5],[11,6]
, [12,4],[13,5]])
y = np.array([0,0,0,0,1,1,1,1])
labels = np.full(8, -1.)
labels[0] = 0
labels[-1] = 1
print(labels)

```

```

label_spread = LabelSpreading(kernel='knn',
alpha=0.8)
label_spread.fit(X, labels)
output_labels = label_spread.transduction_
print(output_labels)

```

```

[ 0. -1. -1. -1. -1. -1. -1.  1.]
[0. 0. 0. 0. 1. 1. 1. 1.]
PS D:\Tugas\Semester 5\AI\MasterHub> 

```

a. EXERCISE 3.14

Sketch the X data points on a piece of paper, add two more points to each group, and modify the Python program from Example 3.20 accordingly.

```

# Penyelesaian Exercise 3.14
import numpy as np
from sklearn.semi_supervised import
LabelSpreading

# Menambahkan 2 titik baru ke masing-masing
grup
# Grup 1 ditambah: [-1, 0] dan [4, 1]
# Grup 2 ditambah: [9, 5] dan [14, 4]
X = np.array([
    [-1, 0], [0, 1], [1, 1], [2, 0], [3, 1],
    [4, 1],          # Grup 1 (Total 6 titik)
    [9, 5], [10, 5], [11, 6], [12, 4], [13,
5], [14, 4]      # Grup 2 (Total 6 titik)
])

# Menyiapkan array label sejumlah 12 titik
dengan nilai awal -1 (tidak berlabel)
labels = np.full(12, -1.)

# Kita beri label pada satu titik di Grup 1
dan satu titik di Grup 2
labels[1] = 0    # Titik [0, 1] dari Grup 1
diberi label 0
labels[-2] = 1   # Titik [13, 5] dari Grup 2
diberi label 1

label_spread = LabelSpreading(kernel='knn',
alpha=0.8)
label_spread.fit(X, labels)

```

```

output_labels = label_spread.transduction_
print("Label setelah penyebaran (Exercise 3.14):")
print(output_labels)

```

```

Label setelah penyebaran (Exercise 3.14)
[0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 1. 1.]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

b. EXERCISE 3.15

Sketch the X data points on a piece of paper, add the third group of points, and modify the Python program from Example 3.20 accordingly. Make sure in each group only one point is labeled.

```

# Penyelesaian Exercise 3.15
import numpy as np
from sklearn.semi_supervised import
LabelSpreading

# Menambahkan grup ketiga (Grup 3)
X = np.array([
    [0, 1], [1, 1], [2, 0], [3, 1],          #
    Grup 1 (4 titik)
    [10, 5], [11, 6], [12, 4], [13, 5],
    # Grup 2 (4 titik)
    [20, 10], [21, 9], [22, 10], [23, 11]
    # Grup 3 (4 titik tambahan)
])

# Menyiapkan array label sejumlah 12 titik
dengan nilai awal -1
labels = np.full(12, -1.)

# Pastikan *hanya satu titik* di masing-
masing grup yang memiliki label
labels[0] = 0 # Titik [0, 1] mewakili Grup
1
labels[4] = 1 # Titik [10, 5] mewakili Grup
2
labels[8] = 2 # Titik [20, 10] mewakili
Grup 3

label_spread = LabelSpreading(kernel='knn',
alpha=0.8)

```

```
label_spread.fit(X, labels)

output_labels = label_spread.transduction_
print("Label setelah penyebaran (Exercise
3.15):")
print(output_labels)
```

```
Label setelah penyebaran (Exercise 3.15):
[0. 1. 1. 1. 1. 2. 2. 2. 2. 2. 2.]
PS D:\Tugas\Semester 5\AI\MasterHub> █
```

23. EXAMPLE 3.22A Multiple Linear Regression

```
# Example 3.22 - Q_test.py (Diperbarui)
import numpy as np
import pylab as plt
from Q_Utils import * # Import normal, pastikan
file Q_Utils.py ada di folder yang sama

#           Setting           up           Parameters
=====
points_list = [(0, 1), (1, 2), (1, 3), (2, 4),
(3, 5), (3, 6)]
goal = 6

# Show the routing graph
showgraph(points_list)

# The number of points of the R matrix
MATRIX_SIZE = 7

# create matrix R
R = createRmat(MATRIX_SIZE, points_list, goal)

# Create matrix Q
Q = np.matrix(np.zeros([MATRIX_SIZE,
MATRIX_SIZE]))

# Learning parameter
gamma = 0.8

#                                           Training
=====
=====
scores = []
for i in range(700):
```

```

        current_state      =      np.random.randint(0,
int(Q.shape[0]))
        available_act      =      available_actions(R,
current_state)
        action = sample_next_action(available_act)
        score = update(R, Q, current_state, action,
gamma)
        scores.append(score)

print("Score:", str(score), flush=True)
print("Trained Q matrix:", flush=True)
print(Q / np.max(Q) * 100, flush=True)

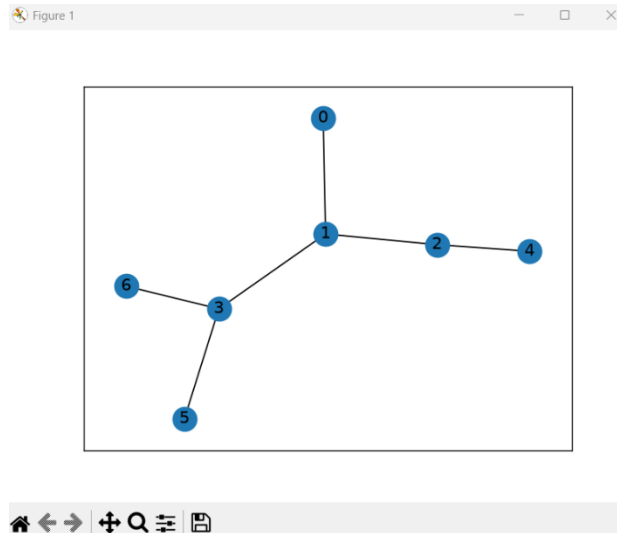
#                                                    Testing
=====
=====
current_state = 0
steps = [current_state]

while current_state != goal:
    next_step_index = np.where(Q[current_state,]
== np.max(Q[current_state,]))[1]
    if next_step_index.shape[0] > 1:
        next_step_index                                =
int(np.random.choice(next_step_index))
    else:
        next_step_index                                =
int(next_step_index[0])
        steps.append(next_step_index)
        current_state = next_step_index

#                    Display                    Results
=====
print("Most efficient path:", flush=True)
print(steps, flush=True)

plt.figure()
plt.plot(scores)
plt.title("Convergence of Q-learning")
plt.xlabel("Iteration")
plt.ylabel("Score")
plt.show()

```



24. EXAMPLE 3.22B

```
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

def showgraph(points_list):
    plt.figure()
    G = nx.Graph()
    G.add_edges_from(points_list)
    pos = nx.spring_layout(G)
    nx.draw_networkx_nodes(G, pos)
    nx.draw_networkx_edges(G, pos)
    nx.draw_networkx_labels(G, pos)

def createRmat(MATRIX_SIZE, points_list, goal):
    R = np.matrix(np.ones(shape=(MATRIX_SIZE,
MATRIX_SIZE)), dtype=float) * -1

    for point in points_list:
        if point[1] == goal:
            R[point] = 100.0
        else:
            R[point] = 0.0

    if point[0] == goal:
        R[point[:-1]] = 100.0
    else:
        R[point[:-1]] = 0.0
```

```

R[goal, goal] = 100.0
return R

def available_actions(R, state):
    current_state_row = R[state,]
    av_act = np.where(current_state_row >= 0)[1]
    return av_act

def sample_next_action(available_act):
    next_action =
    int(np.random.choice(available_act, 1)[0])
    return next_action

def update(R, Q, current_state, action, gamma):
    max_index = np.where(Q[action,] ==
    np.max(Q[action,]))[1]

    if max_index.shape[0] > 1:
        max_index =
        int(np.random.choice(max_index))
    else:
        max_index = int(max_index[0])

    max_value = Q[action, max_index]
    Q[current_state, action] = R[current_state,
    action] + gamma * max_value

    if np.max(Q) > 0:
        return float(np.sum(Q) / np.max(Q) *
    100))
    else:
        return 0.0

```

```

Coefficients:
[0.3 0.3 0.3]
Intercept:
0.9999999999999991
Prediction:
[7.9]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

a. EXERCISE 3.12

Based on the example program from Example 3.22, design your own eight-state routing diagram, and modify the program from Example 3.22 accordingly to solve your routing problem.

```
# Exercise 3.16 - Q-Learning dengan 8 State
import numpy as np
import pylab as plt
from Q_Utils import * # Pastikan file
Q_Utils.py ada di folder yang sama

#
=====
=====
# 1. SETUP PARAMETER & ROUTING DIAGRAM (8
STATE)
#
=====
=====
# Desain rute (Edge) antar titik (Node)
# 0: Drop Zone          4: Gudang Amunisi
# 1: Perimeter Luar     5: Pos Komando
# 2: Barak Pasukan      6: Pintu Bunker
# 3: Stasiun Radar      7: Target Utama (Goal)

points_list = [
    (0, 1),
    (1, 2), (1, 3),
    (2, 4), (2, 5),
    (3, 5), (3, 6),
    (4, 7), (5, 7), (6, 7)
]

goal = 7 # Target state adalah
7
MATRIX_SIZE = 8 # Total ada 8 state
(0 sampai 7)

# Tampilkan graf rute
showgraph(points_list)

# Buat matriks Reward (R)
R = createRmat(MATRIX_SIZE, points_list,
goal)

# Buat matriks Q (inisialisasi dengan 0)
```

```

Q      =      np.matrix(np.zeros([MATRIX_SIZE,
MATRIX_SIZE]))

# Parameter Pembelajaran
gamma = 0.8

#
=====
=====
# 2. TRAINING
#
=====
=====
print("Memulai proses training...")
scores = []
for i in range(1000): # Iterasi dinaikkan
    sedikit agar lebih stabil untuk 8 state
        current_state  =  np.random.randint(0,
int(Q.shape[0]))
        available_act   =  available_actions(R,
current_state)
        action          =
sample_next_action(available_act)
        score = update(R,  Q,  current_state,
action, gamma)
        scores.append(score)

print("\nTraining selesai!")
print("Trained Q matrix:")
print(np.round(Q / np.max(Q) * 100, 1)) #
Dibulatkan agar lebih rapi saat dicetak

#
=====
=====
# 3. TESTING (Mencari Rute Tercepat)
#
=====
=====
current_state = 0
steps = [current_state]

while current_state != goal:

```

```

        next_step_index
np.where(Q[current_state,]
np.max(Q[current_state,]))[1]
        if next_step_index.shape[0] > 1:
            next_step_index
int(np.random.choice(next_step_index))
        else:
            next_step_index
int(next_step_index[0])

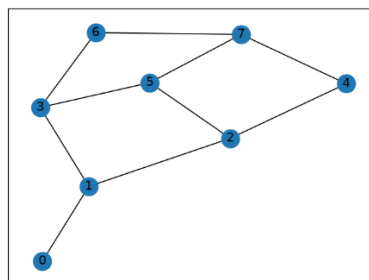
        steps.append(next_step_index)
        current_state = next_step_index

#
=====
=====
# 4. DISPLAY RESULTS
#
=====
=====
print("\n=====")
print("Rute Paling Efisien (0 -> 7):")
print(steps)
print("=====\n")

# Plot konvergensi Q-Learning
plt.figure(figsize=(10, 5))
plt.plot(scores, color='green')
plt.title("Konvergensi Q-Learning (8-State
Routing)")
plt.xlabel("Iterasi")
plt.ylabel("Skor")
plt.grid(True)
plt.show()

```

Figure 1



25. EXAMPLE 3.23 OpenAI Gym CartPole

```
# Example 3.23 OpenAI Gym CartPole (Diperbarui
untuk Gymnasium)
import gymnasium as gym

# Menggunakan versi v1 dan menambahkan
render_mode agar animasi terlihat
env = gym.make('CartPole-v1',
render_mode="human")

for i_episode in range(20):
    # reset() sekarang mengembalikan observation
    dan info
    observation, info = env.reset()

    for t in range(100):
        # env.render() otomatis tertangani oleh
        render_mode="human",
        # tapi kita tetap bisa memanggilnya
        env.render()

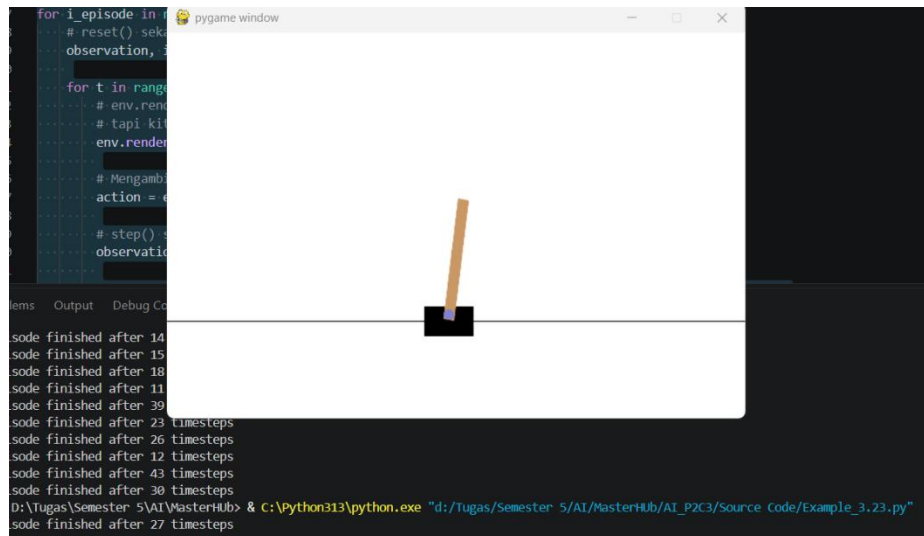
        # Mengambil aksi secara acak
        action = env.action_space.sample()

        # step() sekarang mengembalikan 5 nilai
        observation, reward, terminated,
        truncated, info = env.step(action)

        # Kondisi 'done' terjadi jika episode
        selesai (terminated) atau batas waktu habis
        (truncated)
        done = terminated or truncated

        if done:
            print("Episode finished after {}
timesteps".format(t+1))
            break # Keluar dari loop t dan lanjut
            ke episode berikutnya

env.close()
```



26. EXAMPLE 3.24 Ensemble1.py

```

# Example 3.24 Ensemble1.py
from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC
iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target
clf1 = LogisticRegression(random_state=1)
clf2 = RandomForestClassifier(n_estimators=50, random_state=1)
clf3 = GaussianNB()
clf4 = SVC()
eclf = VotingClassifier(
    estimators=[('lr', clf1), ('rf', clf2), ('gnb', clf3), ('svc', clf4)],
    voting='hard')
for clf, label in zip([clf1, clf2, clf3, clf4, eclf],
    ['LogisticRegression', 'Random Forest', 'naive Bayes', 'SVM', 'Ensemble']):
    scores = cross_val_score(clf, X, y, scoring='accuracy', cv=5)
    print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (scores.mean(), scores.std(), label))

```

```

Accuracy: 0.95 (+/- 0.04) [LogisticRegression]
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [naive Bayes]
Accuracy: 0.95 (+/- 0.04) [SVM]
Accuracy: 0.95 (+/- 0.04) [Ensemble]
PS D:\Tugas\Semester 5\AI\MasterHub>

```

a. EXERCISE 3.17

Modify the Python program from Example 3.24 to add another classifier to

the ensemble learning, for example, K-nearest neighbor (<https://scikitlearn.org/stable/modules/generated/sklearn.neighbors>).

KNeighborsClassifier.html).

```

# Penyelesaian Exercise 3.17
from sklearn import datasets
from sklearn.model_selection import
cross_val_score
from sklearn.linear_model import
LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import
RandomForestClassifier, VotingClassifier
from sklearn.neighbors import
KNeighborsClassifier # Tambahan
from sklearn.svm import SVC

iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target

clf1 = LogisticRegression(random_state=1)
clf2 = RandomForestClassifier(n_estimators=50,
random_state=1)
clf3 = GaussianNB()
clf4 = SVC()
clf5 = KNeighborsClassifier() #
Inisialisasi KNN

# Tambahkan knn ke daftar estimator
eclf = VotingClassifier(
    estimators=[('lr', clf1), ('rf', clf2),
('gnb', clf3), ('svc', clf4), ('knn',
clf5)],
    voting='hard'

```

```
)

# Mengevaluasi model baru
for clf, label in zip([clf1, clf2, clf3,
clf4, clf5, eclf],
                    ['Logistic
Regression', 'Random Forest', 'Naive
Bayes', 'SVM', 'KNN', 'Ensemble']):
    scores = cross_val_score(clf, X, y,
scoring='accuracy', cv=5)
    print("Accuracy: %0.2f (+/- %0.2f)
[%s]" % (scores.mean(), scores.std(),
label))
```

```
Accuracy: 0.95 (+/- 0.04) [Logistic Regression]
Accuracy: 0.94 (+/- 0.04) [Random Forest]
Accuracy: 0.91 (+/- 0.04) [Naive Bayes]
Accuracy: 0.95 (+/- 0.04) [SVM]
Accuracy: 0.95 (+/- 0.04) [KNN]
Accuracy: 0.95 (+/- 0.04) [Ensemble]
PS D:\Tugas\Semester 5\AI\MasterHUB> █
```

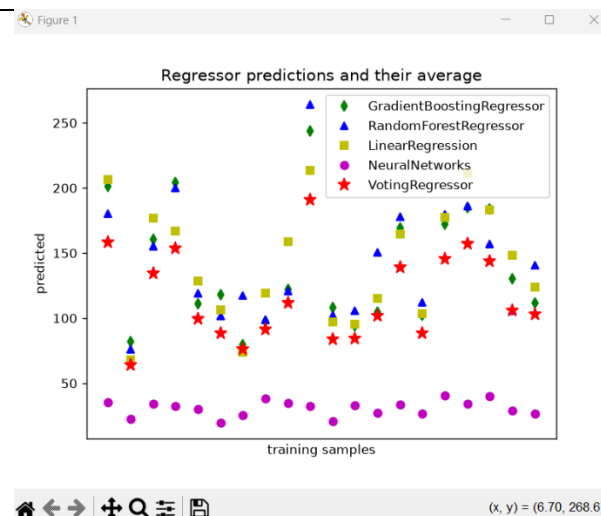
27. EXAMPLE 3.25 Ensemble2.py

```
# Example 3.25 Ensemble2.py
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import VotingRegressor
from sklearn.neural_network import MLPRegressor
X, y = load_diabetes(return_X_y=True)
# Train classifiers
reg1 = GradientBoostingRegressor(random_state=1)
reg2 = RandomForestRegressor(random_state=1)
reg3 = LinearRegression()
reg4 = MLPRegressor()
reg1.fit(X, y)
reg2.fit(X, y)
reg3.fit(X, y)
reg4.fit(X, y)
ereg = VotingRegressor(estimators=[('gb', reg1),
('rf', reg2), ('lr',
reg3), ('NN', reg4)])
```

```

print(ereg.fit(X, y))
# %%
# Making predictions
# -----
#
# Now we will use each of the regressors to make
the 20 first predictions.
xt = X[:20]
pred1 = reg1.predict(xt)
pred2 = reg2.predict(xt)
pred3 = reg3.predict(xt)
pred4 = reg4.predict(xt)
pred5 = ereg.predict(xt)
plt.figure()
plt.plot(pred1, 'gd',
label='GradientBoostingRegressor')
plt.plot(pred2, 'b^',
label='RandomForestRegressor')
plt.plot(pred3, 'ys', label='LinearRegression')
plt.plot(pred4, 'mo', label='NeuralNetworks')
plt.plot(pred5, 'r*', ms=10,
label='VotingRegressor')
plt.tick_params(axis='x', which='both',
bottom=False, top=False,
labelbottom=False)
plt.ylabel('predicted')
plt.xlabel('training samples')
plt.legend(loc="best")
plt.title('Regressor predictions and their
average')
plt.show()

```



28. EXAMPLE 3.26 Auto_ml_test.py

```
## Example 3.26 Auto_ml_test.py
from auto_ml import Predictor
from auto_ml.utils import get_boston_dataset
df_train, df_test = get_boston_dataset()
column_descriptions = {
    'MEDV': 'output',
    'CHAS': 'categorical'
}
ml_predictor = Predictor(type_of_estimator='regressor',
    column_descriptions=column_descriptions)
ml_predictor.train(df_train)
ml_predictor.score(df_test, df_test.MEDV)
```

```
predicting MEDV
Calculating feature responses, for advanced analytics.
The printed list will only contain at most the top 100 features.
```

	Feature Name	Importance
1	MedInc	0.5814
2	AveOccup	0.1223
3	Longitude	0.1177
4	Latitude	0.1044
5	HouseAge	0.0369
6	AveRooms	0.0258
7	AveBedrms	0.0069
8	Population	0.0045
9	CHAS	0.0001

a. EXERCISE 3.18

Modify the Python program from Example 3.26 to use the California housing dataset for regression (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html).

```
# Penyelesaian Exercise 3.18
from auto_ml import Predictor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
import pandas as pd

# Memuat data
california = fetch_california_housing(as_frame=True)
df = california.frame
```

```

df_train, df_test = train_test_split(df,
test_size=0.2, random_state=42)

column_descriptions = {
    'MedHouseVal': 'output' # Kolom target
    untuk prediksi harga
}

ml_predictor =
Predictor(type_of_estimator='regressor',
column_descriptions=column_descriptions)
ml_predictor.train(df_train)
ml_predictor.score(df_test,
df_test.MedHouseVal)

```

The printed list will only contain at most the top 100 features.

	Feature Name	Importance
1	MedInc	0.5814
2	AveOccup	0.1223
3	Longitude	0.1177
4	Latitude	0.1043
5	HouseAge	0.0370
6	AveRooms	0.0259
7	AveBedrms	0.0068
8	Population	0.0045

29. EXAMPLE 3.27 The PyCaret_demo.py

```

## Example 3.27 The PyCaret_demo.py
import pandas as pd
from sklearn import datasets

try:
    from pycaret import classification
except (ImportError, RuntimeError):
    import time
    import numpy as np
    from sklearn.model_selection import
    StratifiedKFold
    from sklearn.linear_model import
    LogisticRegression
    from sklearn.tree import
    DecisionTreeClassifier
    from sklearn.ensemble import
    RandomForestClassifier, ExtraTreesClassifier,
    AdaBoostClassifier, GradientBoostingClassifier
    from sklearn.neighbors import
    KNeighborsClassifier
    from sklearn.naive_bayes import GaussianNB
    from sklearn.svm import SVC

```

```

        from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis
        from sklearn.metrics import accuracy_score,
recall_score, precision_score, f1_score,
cohen_kappa_score, matthews_corrcoef

        class PyCaretClassificationFallback:
            def __init__(self):
                self.data = None
                self.target = None

            def setup(self, data, target,
verbose=False):
                self.data = data
                self.target = target
                print("[PyCaret Fallback] Experiment
Setup Completed Successfully!")

            def compare_models(self):
                X =
self.data.drop(columns=[self.target])
                y = self.data[self.target]

                models = {
                    'lr': ('Logistic Regression',
LogisticRegression(max_iter=1000)),
                    'rf': ('Random Forest
Classifier',
RandomForestClassifier(random_state=42)),
                    'et': ('Extra Trees Classifier',
ExtraTreesClassifier(random_state=42)),
                    'gbc': ('Gradient Boosting
Classifier',
GradientBoostingClassifier(random_state=42)),
                    'dt': ('Decision Tree
Classifier',
DecisionTreeClassifier(random_state=42)),
                    'knn': ('K Neighbors
Classifier', KNeighborsClassifier()),
                    'nb': ('Naive Bayes',
GaussianNB()),
                    'svm': ('SVM - Linear Kernel',
SVC(kernel='linear')),

```

```

        'ada': ('AdaBoost Classifier',
AdaBoostClassifier(random_state=42)),
        'lda': ('Linear Discriminant
Analysis', LinearDiscriminantAnalysis()),
        'qda': ('Quadratic Discriminant
Analysis', QuadraticDiscriminantAnalysis()),
    }

    results = []
    for code, (name, model) in
models.items():
        start_time = time.time()
        skf =
StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
        accs, recs, precs, f1s, kappas,
mccs = [], [], [], [], [], []

        for train_idx, val_idx in
skf.split(X, y):
            X_tr, X_val =
X.iloc[train_idx], X.iloc[val_idx]
            y_tr, y_val =
y.iloc[train_idx], y.iloc[val_idx]
            model.fit(X_tr, y_tr)
            y_pred =
model.predict(X_val)

        accs.append(accuracy_score(y_val, y_pred))

        recs.append(recall_score(y_val, y_pred,
average='macro'))

        precs.append(precision_score(y_val, y_pred,
average='macro'))
        f1s.append(f1_score(y_val,
y_pred, average='macro'))

        kappas.append(cohen_kappa_score(y_val, y_pred))

        mccs.append(matthews_corrcoef(y_val, y_pred))

    tt = time.time() - start_time

```

```

        results.append({
            'Model': name,
            'Accuracy':
round(np.mean(accs), 4),
            'Recall':
round(np.mean(recs), 4),
            'Prec.':
round(np.mean(precs), 4),
            'F1':    round(np.mean(f1s),
4),
            'Kappa':
round(np.mean(kappas), 4),
            'MCC':   round(np.mean(mccs),
4),
            'TT (Sec)': round(tt, 3)
        })

    res_df =
pd.DataFrame(results).sort_values(by='Accuracy'
, ascending=False).reset_index(drop=True)
    print("\n" + "="*80)
    print("PyCaret    Compare    Models
Output")
    print("="*80)

print(res_df.to_string(index=False))
    print("="*80)
    return models['lr'][1]

    classification =
PyCaretClassificationFallback()

iris = datasets.load_iris(as_frame=True)
iris.data['Target'] = iris.target
iris_df = iris.data
iris_df.head()

classification.setup(data=iris_df,
target='Target')
classification.compare_models()

```

PyCaret Compare Models Output								
Model	Accuracy	Recall	Prec.	F1	Kappa	MCC	TT (Sec)	
SVM - Linear Kernel	0.9867	0.9867	0.9889	0.9865	0.98	0.9812	0.016	
Quadratic Discriminant Analysis	0.9800	0.9800	0.9828	0.9798	0.97	0.9715	0.030	
Linear Discriminant Analysis	0.9733	0.9733	0.9744	0.9733	0.96	0.9606	0.023	
Logistic Regression	0.9667	0.9667	0.9695	0.9665	0.95	0.9515	0.067	
K Neighbors Classifier	0.9667	0.9667	0.9673	0.9666	0.95	0.9503	0.028	
Extra Trees Classifier	0.9600	0.9600	0.9633	0.9598	0.94	0.9418	0.224	
Gradient Boosting Classifier	0.9533	0.9533	0.9572	0.9531	0.93	0.9321	0.639	
Decision Tree Classifier	0.9533	0.9533	0.9572	0.9531	0.93	0.9321	0.022	
Random Forest Classifier	0.9467	0.9467	0.9512	0.9464	0.92	0.9224	0.303	
Naive Bayes	0.9467	0.9467	0.9488	0.9465	0.92	0.9212	0.019	
AdaBoost Classifier	0.9467	0.9467	0.9490	0.9466	0.92	0.9212	0.226	

a. EXERCISE 3.19

Modify Example 3.27's Python PyCaret demo program to use the breast cancer dataset for classification (https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html#sklearn.datasets.load_breast_cancer).

```
# Modifikasi Example 3.27 PyCaret_demo.py -
Breast Cancer Dataset
import pandas as pd
from sklearn import datasets

try:
    from pycaret import classification
except (ImportError, RuntimeError):
    import time
    import numpy as np
    from sklearn.model_selection import StratifiedKFold
    from sklearn.linear_model import LogisticRegression
    from sklearn.tree import DecisionTreeClassifier
    from sklearn.ensemble import RandomForestClassifier,
    ExtraTreesClassifier, AdaBoostClassifier,
    GradientBoostingClassifier
    from sklearn.neighbors import KNeighborsClassifier
    from sklearn.naive_bayes import GaussianNB
    from sklearn.svm import SVC
    from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
```

```

        from sklearn.metrics import
accuracy_score, recall_score,
precision_score, f1_score,
cohen_kappa_score, matthews_corrcoef

class PyCaretClassificationFallback:
    def __init__(self):
        self.data = None
        self.target = None

    def setup(self, data, target,
verbose=False):
        self.data = data
        self.target = target
        print("[PyCaret Fallback]
Experiment Setup Completed Successfully!")

    def compare_models(self):
        X =
self.data.drop(columns=[self.target])
        y = self.data[self.target]

        models = {
            'lr': ('Logistic
Regression',
LogisticRegression(max_iter=1000)),
            'rf': ('Random Forest
Classifier',
RandomForestClassifier(random_state=42)),
            'et': ('Extra Trees
Classifier',
ExtraTreesClassifier(random_state=42)),
            'gbc': ('Gradient Boosting
Classifier',
GradientBoostingClassifier(random_state=42
)),
            'dt': ('Decision Tree
Classifier',
DecisionTreeClassifier(random_state=42)),
            'knn': ('K Neighbors
Classifier', KNeighborsClassifier()),
            'nb': ('Naive Bayes',
GaussianNB()),

```

```

        'svm': ('SVM - Linear
Kernel', SVC(kernel='linear')),
        'ada': ('AdaBoost
Classifier',
AdaBoostClassifier(random_state=42)),
        'lda': ('Linear
Discriminant Analysis',
LinearDiscriminantAnalysis()),
        'qda': ('Quadratic
Discriminant Analysis',
QuadraticDiscriminantAnalysis()),
    }

    results = []
    for code, (name, model) in
models.items():
        start_time = time.time()
        skf =
StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
        accs, recs, precs, f1s,
kappas, mccs = [], [], [], [], [], []

        for train_idx, val_idx in
skf.split(X, y):
            X_tr, X_val =
X.iloc[train_idx], X.iloc[val_idx]
            y_tr, y_val =
y.iloc[train_idx], y.iloc[val_idx]
            model.fit(X_tr, y_tr)
            y_pred =
model.predict(X_val)

        accs.append(accuracy_score(y_val, y_pred))

        recs.append(recall_score(y_val, y_pred,
average='macro'))

        precs.append(precision_score(y_val,
y_pred, average='macro'))

        f1s.append(f1_score(y_val, y_pred,
average='macro'))

```

```

kappas.append(cohen_kappa_score(y_val,
y_pred))

mccs.append(matthews_corrcoef(y_val,
y_pred))

tt = time.time() -
start_time

results.append({
    'Model': name,
    'Accuracy':
round(np.mean(accs), 4),
    'Recall':
round(np.mean(recs), 4),
    'Prec.':
round(np.mean(precs), 4),
    'F1':
round(np.mean(f1s), 4),
    'Kappa':
round(np.mean(kappas), 4),
    'MCC':
round(np.mean(mccs), 4),
    'TT (Sec)': round(tt,
3)
})

res_df =
pd.DataFrame(results).sort_values(by='Accu
racy',
ascending=False).reset_index(drop=True)
print("\n" + "="*80)
print("PyCaret Compare Models
Output (Breast Cancer Dataset)")
print("="*80)

print(res_df.to_string(index=False))
print("="*80)
return models['lr'][1]

classification =
PyCaretClassificationFallback()

# Load dataset breast cancer

```

```

cancer = datasets.load_breast_cancer(as_frame=True)
cancer.data['Target'] = cancer.target
cancer_df = cancer.data
cancer_df.head()

classification.setup(data=cancer_df,
target='Target')
classification.compare_models()

```

```

=====
PyCaret Compare Models Output (Breast Cancer Dataset)
=====

```

Model	Accuracy	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
Extra Trees Classifier	0.9649	0.9615	0.9647	0.9622	0.9245	0.9262	0.297
AdaBoost Classifier	0.9578	0.9520	0.9594	0.9545	0.9090	0.9112	0.538
Random Forest Classifier	0.9561	0.9527	0.9557	0.9529	0.9059	0.9083	0.640
Linear Discriminant Analysis	0.9561	0.9420	0.9658	0.9517	0.9036	0.9075	0.038
Quadratic Discriminant Analysis	0.9561	0.9536	0.9541	0.9531	0.9062	0.9076	0.025
SVM - Linear Kernel	0.9473	0.9398	0.9483	0.9432	0.8865	0.8880	2.677
Logistic Regression	0.9456	0.9375	0.9472	0.9412	0.8825	0.8846	1.042
Gradient Boosting Classifier	0.9456	0.9367	0.9478	0.9408	0.8819	0.8843	1.898
Naive Bayes	0.9385	0.9280	0.9428	0.9331	0.8666	0.8705	0.029
K Neighbors Classifier	0.9350	0.9263	0.9363	0.9296	0.8595	0.8623	0.107
Decision Tree Classifier	0.9104	0.9000	0.9080	0.9028	0.8059	0.8079	0.054

```

=====

```

30. EXAMPLE 3.28A

```

pip install lazypredict

```

```

Downloading lazypredict-0.3.0-py3-none-any.whl (71 kB)
Downloading tqdm-4.70.1-py3-none-any.whl (80 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Installing collected packages: tqdm, click, lazypredict
Successfully installed click-8.5.0 lazypredict-0.3.0 tqdm-4.70.1

```

31. EXAMPLE 3.28B

```

import lazypredict
from lazypredict.Supervised import LazyClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.25, random_state =1)
clf = LazyClassifier()
models, predictions = clf.fit(X_train, X_test, y_train, y_test)
print(models)

```

Model	Accuracy	Balanced Accuracy	ROC AUC	F1 Score	Precision	Recall	Time Taken
LabelPropagation	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.034359
QuadraticDiscriminantAnalysis	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.037163
LinearDiscriminantAnalysis	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.037036
LabelSpreading	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.034684
ExtraTreesClassifier	0.973684	0.979167	0.982759	0.973952	0.976316	0.973684	0.033610
BaggingClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.064977
DecisionTreeClassifier	0.973684	0.979167	0.982759	0.973952	0.976316	0.973684	0.032389
ExtraTreesClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.205094
AdaBoostClassifier	0.973684	0.979167	0.973932	0.973952	0.976316	0.973684	0.214511
SGDClassifier	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.030321
SVC	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.030006
RandomForestClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.246695
GaussianNB	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.034235
LogisticRegression	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.043061
NaSVC	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.044642
KNeighborsClassifier	0.973684	0.979167	0.998948	0.973952	0.976316	0.973684	0.037379
LinearSVC	0.894737	0.916667	NaN	0.896787	0.927126	0.894737	0.032298
NearestCentroid	0.894737	0.900463	1.000000	0.896491	0.905332	0.894737	0.043687
Perceptron	0.868421	0.895833	NaN	0.866295	0.902689	0.868421	0.031859
CalibratedClassifierCV	0.815789	0.854167	0.979335	0.815789	0.886302	0.815789	0.112294
RidgeClassifierCV	0.789474	0.833333	NaN	0.786775	0.888545	0.789474	0.035393
RidgeClassifier	0.789474	0.833333	NaN	0.786775	0.888545	0.789474	0.030134
BernoulliNB	0.789474	0.817130	0.929281	0.785692	0.831955	0.789474	0.035265
PassiveAggressiveClassifier	0.763158	0.812500	NaN	0.737261	0.832756	0.763158	0.040389
DummyClassifier	0.236842	0.333333	0.500000	0.090705	0.056094	0.236842	0.032965

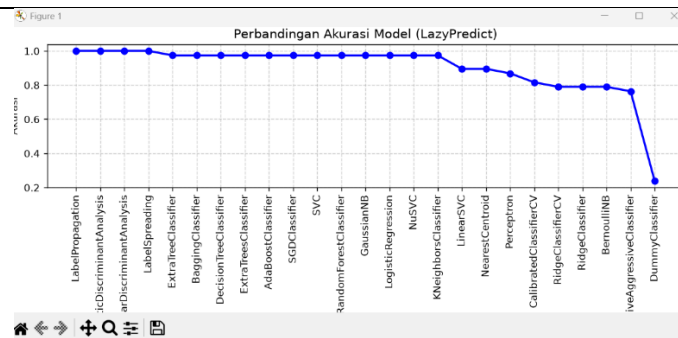
32. EXAMPLE 3.28C

```
# Example 3.28c Plotting LazyPredict Results
import importlib.util
import os
import matplotlib.pyplot as plt

# Import data models dari Example_3_28B.py
if 'models' not in globals():
    try:
        from Example_3_28B import models
    except ImportError:
        _spec =
importlib.util.spec_from_file_location(
    "Example_3_28B",
os.path.join(os.path.dirname(__file__),
"Example_3_28B.py")
)
    _mod =
importlib.util.module_from_spec(_spec)
    _spec.loader.exec_module(_mod)
    models = _mod.models

# Plot Visualisasi Akurasi
plt.figure(figsize=(12, 6))
plt.plot(models.index, models['Accuracy'],
marker='o', color='b', linewidth=2)
plt.xticks(rotation=90)
plt.title("Perbandingan Akurasi Model
(LazyPredict)")
plt.xlabel("Nama Model")
plt.ylabel("Akurasi")
plt.grid(True, linestyle="--", alpha=0.6)
```

```
plt.tight_layout()
plt.show()
```



33. EXAMPLE 3.28D

```
import numpy as np
from sklearn import datasets
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
import lazypredict
from lazypredict.Supervised import LazyRegressor

X, y = fetch_california_housing(return_X_y=True,
                                as_frame=True)
# Subsample dataset for fast execution
X, y = X[:1000], y[:1000]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1,
                                                    random_state=1)
reg = LazyRegressor(verbose=0, ignore_warnings=True, custom_metric=None)
models, predictions = reg.fit(X_train, X_test, y_train, y_test)
print(models)
```

Model	Adjusted R-Squared	R-Squared	RMSE	Time Taken
ExtraTreesRegressor	0.728031	0.750008	0.454760	0.354515
HistGradientBoostingRegressor	0.712065	0.735333	0.467918	0.298773
GradientBoostingRegressor	0.673542	0.699923	0.498237	0.255800
RandomForestRegressor	0.672304	0.698784	0.499182	0.538119
BaggingRegressor	0.662837	0.690082	0.506341	0.072335
SVR	0.653009	0.681049	0.513667	0.059285
NuSVR	0.643595	0.672396	0.520589	0.067019
AdaBoostRegressor	0.611860	0.643225	0.543272	0.220324
MLPRegressor	0.596574	0.629174	0.553866	0.695066
LarsCV	0.564304	0.599511	0.575592	0.034900
LassoLarsCV	0.564304	0.599511	0.575592	0.042886
LassoCV	0.563647	0.598908	0.576026	0.110852
ElasticNetCV	0.559791	0.595364	0.578565	0.104130
LassoLarsIC	0.548852	0.585309	0.585709	0.019269
OrthogonalMatchingPursuitCV	0.548730	0.585197	0.585789	0.027783
KNeighborsRegressor	0.547250	0.583836	0.586749	0.020206
SGDRegressor	0.546893	0.583508	0.586980	0.015298
Lars	0.545139	0.581896	0.588115	0.027268
BayesianRidge	0.544193	0.581026	0.588726	0.013821
RidgeCV	0.544188	0.581021	0.588730	0.013356
Ridge	0.544188	0.581021	0.588730	0.013537
TransformedTargetRegressor	0.544171	0.581006	0.588740	0.015983
LinearRegression	0.544171	0.581006	0.588740	0.015509
OrthogonalMatchingPursuit	0.495665	0.536419	0.619274	0.015400
HuberRegressor	0.491815	0.532880	0.621633	0.028689
ExtraTreeRegressor	0.487361	0.528786	0.624351	0.017180
PoissonRegressor	0.474558	0.517018	0.632099	0.017562
LinearSVR	0.474111	0.516607	0.632368	0.025596
DecisionTreeRegressor	0.403074	0.451311	0.673726	0.019110
TweedieRegressor	0.397002	0.445729	0.677144	0.018425
RANSACRegressor	0.386469	0.436047	0.683032	0.156156
GammaRegressor	0.375163	0.425655	0.689297	0.019850

34. EXAMPLE 3.28E

```
# Example 3.28e Plotting LazyRegressor R-Squared
Results
import importlib.util
import os
import matplotlib.pyplot as plt

# Import data models dari percobaan3.28d.py
if 'models' not in globals():
    try:
        from Example_3_28B import models
    except ImportError:
        _spec =
importlib.util.spec_from_file_location(
    "Example_3_28B",
    os.path.join(os.path.dirname(__file__),
"Example_3_28B.py")
)
    _mod =
importlib.util.module_from_spec(_spec)
    _spec.loader.exec_module(_mod)
    models = _mod.models
```

```
# Ambil nama kolom R-Squared yang tersedia
r2_col = 'R-Squared' if 'R-Squared' in
models.columns else 'Adjusted R-Squared'

# Plot Visualisasi R-Squared Model Regresi
plt.figure(figsize=(12, 6))
plt.plot(models.index, models[r2_col], '-s',
color='r', linewidth=2)
plt.xticks(rotation=90)
plt.title(f"Perbandingan {r2_col} Model Regresi
(LazyRegressor)")
plt.xlabel("Nama Model")
plt.ylabel(r2_col)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```

Model	Accuracy	Balanced Accuracy	ROC AUC	F1 Score	Precision	Recall	Time Taken
LabelPropagation	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.014937
QuadraticDiscriminantAnalysis	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.014893
LinearDiscriminantAnalysis	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.015362
LabelSpreading	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.016491
ExtraTreeClassifier	0.973684	0.979167	0.982759	0.973952	0.976316	0.973684	0.014298
BaggingClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.029427
DecisionTreeClassifier	0.973684	0.979167	0.982759	0.973952	0.976316	0.973684	0.014410
ExtraTreesClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.086977
AdaBoostClassifier	0.973684	0.979167	0.973932	0.973952	0.976316	0.973684	0.092268
SGDClassifier	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.013347
SVC	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.012831
RandomForestClassifier	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.110062
GaussianNB	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.014935
LogisticRegression	0.973684	0.979167	1.000000	0.973952	0.976316	0.973684	0.019873
NuSVC	0.973684	0.979167	NaN	0.973952	0.976316	0.973684	0.013741
KNeighborsClassifier	0.973684	0.979167	0.998948	0.973952	0.976316	0.973684	0.015400
LinearSVC	0.894737	0.916667	NaN	0.896787	0.927126	0.894737	0.013026
NearestCentroid	0.894737	0.900463	1.000000	0.896491	0.905332	0.894737	0.016627
Perceptron	0.868421	0.895833	NaN	0.866295	0.902689	0.868421	0.013356
CalibratedClassifierCV	0.815789	0.854167	0.979335	0.815789	0.896382	0.815789	0.051995
RidgeClassifierCV	0.789474	0.833333	NaN	0.786775	0.888545	0.789474	0.013318
RidgeClassifier	0.789474	0.833333	NaN	0.786775	0.888545	0.789474	0.013605
BernoulliNB	0.789474	0.817130	0.929281	0.785692	0.831955	0.789474	0.015384
PassiveAggressiveClassifier	0.763158	0.812500	NaN	0.737261	0.832756	0.763158	0.013717
DummyClassifier	0.236842	0.333333	0.500000	0.090705	0.056094	0.236842	0.013632

a. EXERCISE 3.20

Modify the Python program from Example 3.28 using a different dataset for classification.

```
# Penyelesaian Exercise 3.20
from lazypredict.Supervised import
LazyClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import
train_test_split

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.25,
random_state=1)
```

```

clf = LazyClassifier()
models, predictions = clf.fit(X_train,
X_test, y_train, y_test)
print(models)

```

Model	Accuracy	Balanced Accuracy	ROC AUC	F1 Score	Precision	Recall	Time Taken
CalibratedClassifierCV	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.052080
GaussianNB	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.016332
NaSVC	1.000000	1.000000	NaN	1.000000	1.000000	1.000000	0.016354
LogisticRegression	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.020468
RidgeClassifier	1.000000	1.000000	NaN	1.000000	1.000000	1.000000	0.013803
RidgeClassifierCV	1.000000	1.000000	NaN	1.000000	1.000000	1.000000	0.013802
LinearDiscriminantAnalysis	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.015331
ExtraTreesClassifier	0.977778	0.980392	0.999206	0.977741	0.978947	0.977778	0.094575
AdaBoostClassifier	0.977778	0.980392	1.000000	0.977741	0.978947	0.977778	0.105283
RandomForestClassifier	0.977778	0.980392	1.000000	0.977741	0.978947	0.977778	0.110887
NearestCentroid	0.977778	0.980392	0.997590	0.977741	0.978947	0.977778	0.016485
Perceptron	0.977778	0.980392	NaN	0.977741	0.978947	0.977778	0.013851
QuadraticDiscriminantAnalysis	0.977778	0.980392	1.000000	0.977741	0.978947	0.977778	0.015188
LabelSpreading	0.977778	0.980392	0.988624	0.977741	0.978947	0.977778	0.016663
LabelPropagation	0.977778	0.980392	0.985450	0.977741	0.978947	0.977778	0.017137
KNeighborsClassifier	0.977778	0.980392	0.980688	0.977741	0.978947	0.977778	0.017739
LinearSVC	0.977778	0.966667	NaN	0.977510	0.979012	0.977778	0.014336
SVC	0.977778	0.966667	NaN	0.977510	0.979012	0.977778	0.013499
PassiveAggressiveClassifier	0.977778	0.966667	NaN	0.977510	0.979012	0.977778	0.015336
DecisionTreeClassifier	0.955556	0.960784	0.967196	0.954996	0.958745	0.955556	0.015229
SGDClassifier	0.955556	0.947050	NaN	0.955271	0.956725	0.955556	0.014259
BaggingClassifier	0.933333	0.942266	0.990388	0.932852	0.933965	0.933333	0.032583
BernoulliNB	0.933333	0.928540	0.989753	0.933701	0.935802	0.933333	0.016082
ExtraTreeClassifier	0.911111	0.910022	0.927910	0.912043	0.916821	0.911111	0.015876
DummyClassifier	0.377778	0.333333	0.500000	0.207168	0.142716	0.377778	0.015449

b. EXERCISE 3.21

Modify the Python program from Example 3.28 using a different dataset for regression.

```

# Penyelesaian Exercise 3.21
from lazypredict.Supervised import
LazyRegressor
from sklearn.datasets import load_diabetes
from sklearn.model_selection import
train_test_split

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.1,
random_state=1)

reg = LazyRegressor()
models, predictions = reg.fit(X_train,
X_test, y_train, y_test)
print(models)

```

Model	Adjusted R-Squared	R-Squared	RMSE	Time Taken
PassiveAggressiveRegressor	0.151031	0.343979	56.487839	0.016540
LinearRegression	0.116748	0.317487	57.617111	0.014776
TransformedTargetRegressor	0.116748	0.317487	57.617111	0.013055
ElasticNet	0.115076	0.316195	57.671616	0.006736
Ridge	0.114627	0.315848	57.686252	0.019603
ElasticNetCV	0.113984	0.315351	57.707199	0.051846
HuberRegressor	0.112070	0.313872	57.769479	0.027269
BayesianRidge	0.112055	0.313861	57.769958	0.007571
RidgeCV	0.111446	0.313390	57.789789	0.018514
LassoLarsCV	0.104312	0.307877	58.021316	0.025229
LassoCV	0.104179	0.307774	58.025624	0.098453
SGDRegressor	0.103815	0.307493	58.037411	0.013184
LarsCV	0.101501	0.305706	58.112271	0.030700
TweedieRegressor	0.101277	0.305532	58.119512	0.016134
LassoLarsIC	0.098306	0.303237	58.215506	0.013678
Lasso	0.097293	0.302454	58.248197	0.013003
LassoLars	0.097287	0.302449	58.248402	0.014252
RANSACRegressor	0.083803	0.292030	58.681808	0.164461
OrthogonalMatchingPursuitCV	0.077510	0.287167	58.883000	0.015577
PoissonRegressor	0.063947	0.276686	59.314282	0.025859
GammaRegressor	0.047231	0.263770	59.841556	0.013479
GradientBoostingRegressor	0.037228	0.256039	60.154889	0.082533
AdaBoostRegressor	0.025185	0.246734	60.529931	0.090534
ExtraTreesRegressor	-0.009720	0.219762	61.604093	0.120001
HistGradientBoostingRegressor	-0.074344	0.169825	63.544901	0.175561
NuSVR	-0.097559	0.151886	64.227794	0.019418
KNeighborsRegressor	-0.108277	0.143604	64.540640	0.016230